CMSC132: Introduction to Graphs

Chapter 13

By Naren Ayinala

May 2026

Contents

1 What is a Graph? 3

1.1 Directed vs Undirected Graphs . . . 3

1.2 Weighted and Unweighted Graphs . . . 3

1.3 Other Useful Graph Terms . . . 4

2 Paths and Connectivity 4

2.1 Paths . . . 4

2.2 Connected Undirected Graphs . . . 5

2.3 Strong and Weak Connectivity in Directed Graphs . . . 5

3 Free Trees 5

3.1 Important Properties of a Free Tree . . . 5

3.2 Free Trees vs General Graphs . . . 6

4 Rooted Trees 6

4.1 Tree Terminology . . . 7

4.2 Binary Trees . . . 7

5 Representing Graphs in Java 7

5.1 Adjacency Matrix . . . 8

5.2 Adjacency List . . . 8

5.3 Which Representation Should I Use? . . . 10

6 Finding Reachable Vertices 10

6.1 Breadth First Search (BFS) . . . 10

6.2 Depth First Search (DFS) . . . 11

7 Common Mistakes to Avoid **11**

8 Quick Practice Questions **12**

9 Summary **12**

1 What is a Graph?

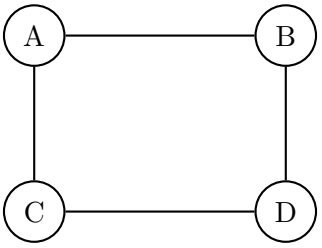
A graph is a mathematical structure used to model relationships between objects. Basically, each object is represented by a **vertex** (also called a **node**), and a connection between two objects is represented by an **edge**. For example, imagine a social network. Every person can be represented as a vertex, while a friendship between two people is represented as an edge.

Formally, we write a graph as

$$G = (V, E)$$

where V is the set of vertices and E is the set of edges. In this chapter we work with finite graphs, meaning both sets are finite.

Example: Let $V = \{A, B, C, D\}$ and $E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}\}$. This is an undirected graph with four vertices and four edges.

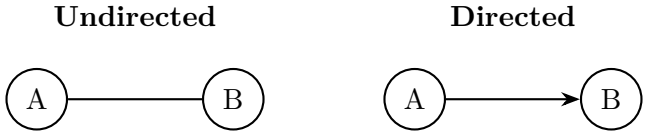


Notice that the position of the nodes on the page does not matter. The graph is determined by which nodes are connected, not by how we draw it.

1.1 Directed vs Undirected Graphs

Undirected graph: The edges have no direction. If A is connected to B, then B is also connected to A. A friendship is a good example if we assume friendships are mutual. An undirected edge can be written as $\{A, B\}$.

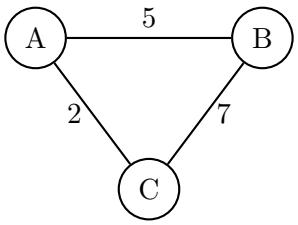
Directed graph: Every edge has a direction. An edge $A \to B$ does not automatically imply an edge $B \to A$. Think of following someone on a social media platform. You might follow them without them following you back. A directed edge is an ordered pair (A, B).



Important: In a directed graph, the arrow direction matters when determining whether one vertex can reach another.

1.2 Weighted and Unweighted Graphs

An **unweighted graph** only tells us whether two vertices are connected. A **weighted graph** also associates a value with each edge. For example, if vertices represent cities, the weight might represent distance, travel time, or cost.



Here, the shortest direct edge by weight is A to C, with weight 2. The number of edges on a path and the total weight of a path are not necessarily the same thing.

1.3 Other Useful Graph Terms

- **Adjacent vertices:** Two vertices directly connected by an edge.
- **Incident edge:** An edge touching a particular vertex.
- **Degree:** In an undirected graph, the number of edges incident to a vertex. A self-loop counts twice toward degree.
- **Indegree:** In a directed graph, the number of edges entering a vertex.
- **Outdegree:** In a directed graph, the number of edges leaving a vertex.
- **Self-loop:** An edge connecting a vertex to itself.
- **Simple graph:** An undirected graph with no self-loops or multiple edges between the same pair of vertices.

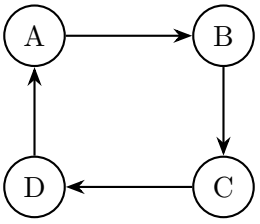
For a finite undirected graph, the sum of all vertex degrees equals $2|E|$, because every edge contributes to two endpoints. For a directed graph, the sum of indegrees and the sum of outdegrees both equal $|E|$.

2 Paths and Connectivity

2.1 Paths

A path is a sequence of vertices where each consecutive pair is connected by an edge. In a directed graph, the path must follow the arrows. For example, $A \to B \to D$ is a path of length 2 because it uses two edges.

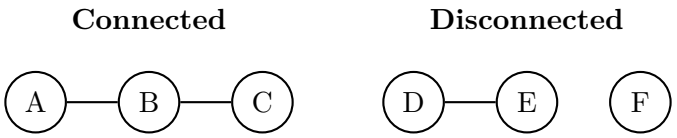
A **simple path** does not repeat any vertices. A **cycle** starts and ends at the same vertex, with no other repeated vertices (for the usual simple cycle definition). A cycle is not normally called a simple path, because its starting vertex is repeated at the end.



Here, $A \to B \to C$ is a simple path, while $A \to B \to C \to D \to A$ is a directed cycle.

2.2 Connected Undirected Graphs

An undirected graph is **connected** if there is a path between every pair of vertices. This does not mean every pair must have a direct edge. It only means we can eventually travel from one to the other by following edges.



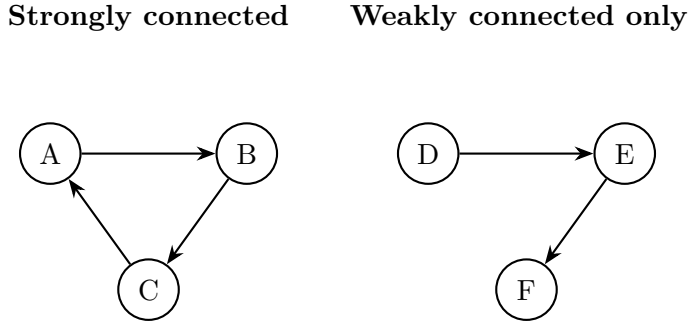
The disconnected graph has two **connected components**: $\{D, E\}$ and $\{F\}$. A component is a maximal connected group of vertices.

2.3 Strong and Weak Connectivity in Directed Graphs

For directed graphs, connectivity becomes slightly more complicated because we have to worry about the direction of the arrows.

Strongly connected: Every vertex can reach every other vertex by following directed edges. In other words, for every pair u, v, there is a directed path from u to v and one from v to u.

Weakly connected: If we ignore the directions of all edges, the resulting undirected graph is connected. A weakly connected graph does not have to be strongly connected.



In the left graph, we can travel around the cycle to get from any node to any other node. In the right graph, F cannot reach D, but the graph becomes connected if we ignore arrow directions.

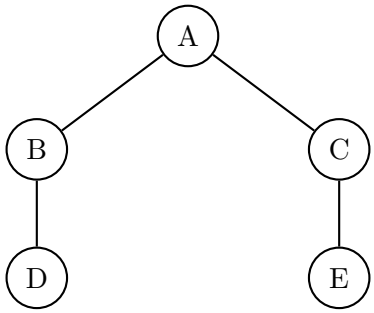
3 Free Trees

A **free tree** is an undirected graph that is both **connected** and **acyclic** (meaning it has no cycles). You can think of a free tree as a graph where all vertices are connected, but there are no unnecessary extra edges creating loops.

3.1 Important Properties of a Free Tree

If a free tree has n vertices, then:

1. It has exactly $n - 1$ edges.
2. There is exactly one simple path between any two distinct vertices.
3. Adding an edge between two previously unconnected-by-an-edge vertices creates a cycle.
4. Removing any existing edge disconnects the tree.



This tree has 5 vertices and 4 edges. Notice there is only one simple path from D to E: $D \to B \to A \to C \to E$. If we add an edge directly between D and E, we create a cycle.

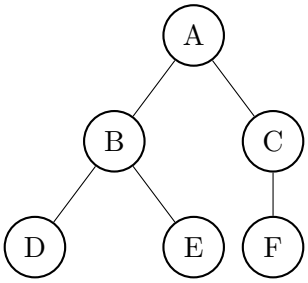
3.2 Free Trees vs General Graphs

Not every graph is a tree. A graph with a cycle is not a free tree, and a disconnected graph is not a free tree either. A graph can also have several disconnected tree components; this is called a **forest**.

4 Rooted Trees

A **rooted tree** is a tree where we choose one vertex to be the **root**. Once we select a root, we get a hierarchy. Every other node has exactly one parent, and a node can have zero or more children. We often draw the root at the top and its children below it.

Clarification: The original free tree edges do not have to become directed graph edges. Instead, the root gives us an implicit parent-to-child orientation.



Here, A is the root. B and C are children of A. D and E are children of B. F is a child of C.

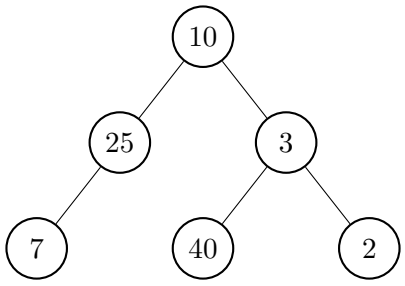
4.1 Tree Terminology

- **Root:** The node with no parent. In the diagram, this is A.
- **Parent:** The node directly above another node. For example, B is the parent of D.
- **Child:** A node directly below its parent. For example, E is a child of B.
- **Siblings:** Nodes with the same parent. For example, D and E are siblings.
- **Interior node:** A node with at least one child. Here, A, B, and C are interior nodes.
- **Leaf node:** A node with no children. Here, D, E, and F are leaves.
- **Subtree:** A node together with all its descendants. The subtree rooted at B contains B, D, and E.
- **Depth:** The number of edges from the root to a particular node. A has depth 0, while D has depth 2.
- **Height of a node:** The maximum number of edges from that node down to a leaf. The height of the root is the height of the entire tree.

Example: The tree above has height 2. A tree consisting of only one root has height 0. Some textbooks count levels or nodes instead of edges, so always check which convention is being used.

4.2 Binary Trees

A **binary tree** is a rooted tree where each node has **at most two children**, usually called the left child and the right child. This means a node can have 0, 1, or 2 children. A binary tree does *not* have to be sorted or balanced.



This is a valid binary tree even though 25 is on the left of 10 and 3 is on the right. A **binary search tree** has additional ordering rules, which we cover in Chapter 14.

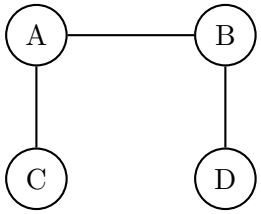
5 Representing Graphs in Java

Up to this point, we have been discussing graphs mathematically. However, when we write a program, we need some way to store vertices and edges in memory. Two common approaches are an **adjacency matrix** and an **adjacency list**. These are extra implementation details added to connect the graph concepts to Java.

5.1 Adjacency Matrix

An adjacency matrix uses a two dimensional array. If there are n vertices, the matrix has n rows and n columns. For an unweighted graph, `matrix[i][j]` is true if there is an edge from vertex i to vertex j.

Consider the undirected graph with edges $A - B$, $A - C$, and $B - D$:



	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	0
D	0	1	0	0

Because the graph is undirected, the matrix is symmetric across its diagonal. For a directed graph, that does not have to be true.

```
public class MatrixGraph {
    private final boolean[][] edges;

    public MatrixGraph(int vertices) {
        edges = new boolean[vertices][vertices];
    }

    public void addUndirectedEdge(int a, int b) {
        edges[a][b] = true;
        edges[b][a] = true;
    }

    public void addDirectedEdge(int from, int to) {
        edges[from][to] = true;
    }

    public boolean hasEdge(int from, int to) {
        return edges[from][to];
    }

    public static void main(String[] args) {
        MatrixGraph graph = new MatrixGraph(4);
        graph.addUndirectedEdge(0, 1);
        graph.addUndirectedEdge(0, 2);
        graph.addUndirectedEdge(1, 3);
        System.out.println(graph.hasEdge(0, 2)); // true
        System.out.println(graph.hasEdge(2, 3)); // false
    }
}
```

Complexity: Space is $O(V^2)$, because we allocate an entry for every possible pair of vertices. Checking whether an edge exists takes $O(1)$ time.

5.2 Adjacency List

Instead of storing every possible pair, an adjacency list stores only the neighbors of each vertex. In Java, we can represent this using an `ArrayList<ArrayList<Integer>>` or a `List<List<Integer>>`.

For the same graph:

Vertex	Neighbors
A	B, C
B	A, D
C	A
D	B

```java
import java.util.ArrayList;
import java.util.List;

public class ListGraph {
    private final List<List<Integer>> neighbors;

    public ListGraph(int vertices) {
        neighbors = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            neighbors.add(new ArrayList<>());
        }
    }

    public void addUndirectedEdge(int a, int b) {
        neighbors.get(a).add(b);
        neighbors.get(b).add(a);
    }

    public void addDirectedEdge(int from, int to) {
        neighbors.get(from).add(to);
    }

    public List<Integer> getNeighbors(int vertex) {
        return List.copyOf(neighbors.get(vertex));
    }

    public static void main(String[] args) {
        ListGraph graph = new ListGraph(4);
        graph.addUndirectedEdge(0, 1);
        graph.addUndirectedEdge(0, 2);
        graph.addUndirectedEdge(1, 3);
        System.out.println(graph.getNeighbors(0)); // [1, 2]
        System.out.println(graph.getNeighbors(1)); // [0, 3]
    }
}
```

Complexity: An adjacency list takes $O(V + E)$ space. Iterating through a vertex's neighbors takes time proportional to its degree (or outdegree in a directed graph). Checking whether an arbitrary edge exists takes $O(\deg(v))$ with an unsorted list, but can be faster if we use a hash set instead.

5.3 Which Representation Should I Use?

Property	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Check edge	$O(1)$	$O(\deg(v))$ with lists
Iterate neighbors	$O(V)$	$O(\deg(v))$
Best for	Dense graphs	Sparse graphs

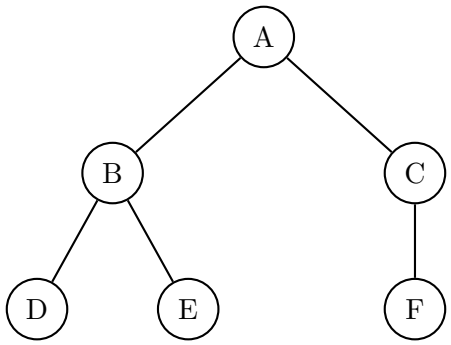
A **dense graph** has many edges relative to the number of possible edges. A **sparse graph** has relatively few edges. Most large real world networks are sparse, so adjacency lists are often a natural choice.

6 Finding Reachable Vertices

Another common question is whether one vertex can reach another. This is where graph traversal comes in. We will study graph algorithms in greater depth in Chapter 17, but here is a useful introduction.

6.1 Breadth First Search (BFS)

BFS explores vertices layer by layer. It uses a **queue**, meaning the first vertex added is the first one removed. In an unweighted graph, BFS can find the minimum number of edges from the starting vertex to every reachable vertex.



Starting from A and processing neighbors left to right, BFS visits A, B, C, D, E, F. First we visit the root, then the nodes one edge away, then the nodes two edges away.

```
import java.util.*;

public class BFSExample {
    public static void bfs(List<List<Integer>> graph, int start) {
        boolean[] visited = new boolean[graph.size()];
        Queue<Integer> queue = new ArrayDeque<>();

        visited[start] = true;
        queue.add(start);

        while (!queue.isEmpty()) {
            int current = queue.remove();
            System.out.print(current + " ");
```

```
            for (int next : graph.get(current)) {
                if (!visited[next]) {
                    visited[next] = true;
                    queue.add(next);
                }
            }
        }
    }
}
```

Why mark visited when adding to the queue? Because otherwise the same vertex might be added several times by different neighbors. Marking it immediately avoids duplicates.

6.2 Depth First Search (DFS)

DFS goes as deep as possible along one branch before backtracking. We can implement it using recursion or an explicit stack. For the same tree, a DFS that visits children from left to right would visit A, B, D, E, C, F.

```
import java.util.List;

public class DFSExample {
    public static void dfs(List<List<Integer>> graph,
                           int current, boolean[] visited) {
        visited[current] = true;
        System.out.print(current + " ");

        for (int next : graph.get(current)) {
            if (!visited[next]) {
                dfs(graph, next, visited);
            }
        }
    }
}
```

Both BFS and DFS take $O(V+E)$ time with adjacency lists, because we process each vertex and examine its adjacency list. We need a `visited` structure in general graphs because cycles could otherwise make us loop forever.

7 Common Mistakes to Avoid

1. **Thinking a path requires a direct edge:** A path may go through several intermediate vertices.
2. **Ignoring arrows in a directed graph:** $A \to B$ does not imply $B \to A$.
3. **Confusing weak and strong connectivity:** Ignoring arrow directions is only allowed when checking weak connectivity.
4. **Thinking every connected graph is a tree:** A connected graph can still have cycles.
5. **Thinking every binary tree is a BST:** A BST must also satisfy an ordering invariant.

6. **Confusing height and depth:** Depth measures distance upward to the root; height measures the longest downward path to a leaf.

7. **Assuming BFS and DFS have one fixed order:** The precise traversal order depends on how neighbors are ordered.

8 Quick Practice Questions

1. A free tree has 12 vertices. How many edges does it have?
   Answer: 11, because a free tree has $n - 1$ edges.

2. If a directed graph has $A \to B$ and $B \to C$, can C necessarily reach A?
   Answer: No. The existing arrows only guarantee a path from A to C.

3. A tree has one root and no other vertices. What is its height?
   Answer: 0, when height counts edges.

4. Is a connected graph with 5 vertices and 5 edges a free tree?
   Answer: No. A free tree with 5 vertices must have exactly 4 edges.

5. What is the difference between a binary tree and a BST?
   Answer: A binary tree has at most two children per node. A BST additionally keeps smaller values in the left subtree and larger values in the right subtree, assuming unique keys.

6. Why might an adjacency list be better for a social network with millions of users?
   Answer: Most users are connected to only a tiny fraction of all users. An adjacency matrix would reserve space for every possible pair, while adjacency lists store the actual connections.

9 Summary

A graph is a collection of vertices and edges used to represent relationships. Graphs may be directed or undirected, weighted or unweighted. Paths describe how vertices can be reached, while connectivity describes whether vertices can reach one another. A free tree is a connected, acyclic, undirected graph. Choosing a root creates a hierarchy, and restricting every node to at most two children gives us a binary tree. Finally, adjacency matrices and adjacency lists let us store graphs in Java, while BFS and DFS let us explore them.